Module 4

UVM Components

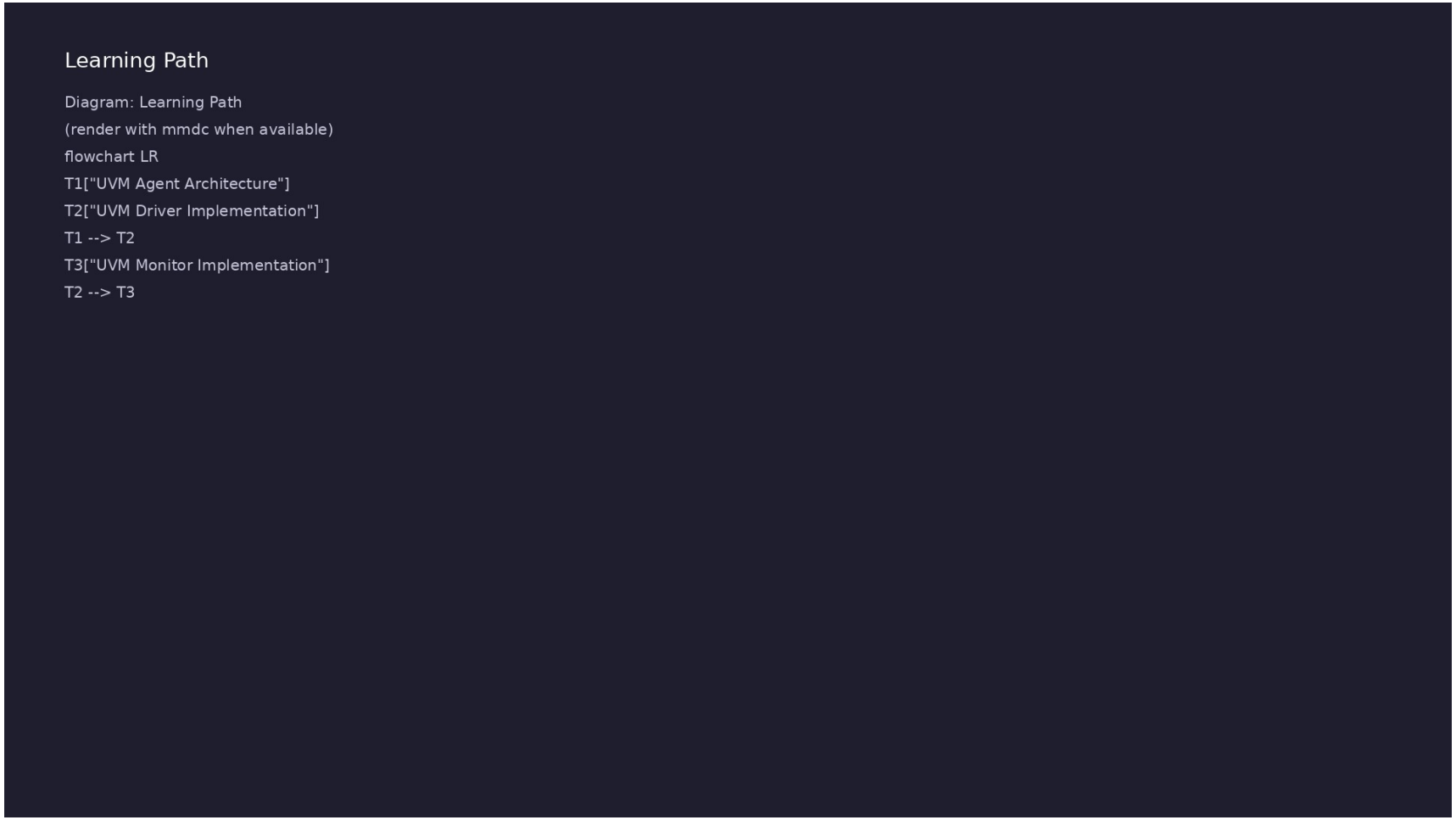
Learning objectives

- Build complete UVM agents with driver, monitor, and sequencer

Prerequisites

- See module README

Learning path



Build complete UVM agents with driver, monitor, and sequencer

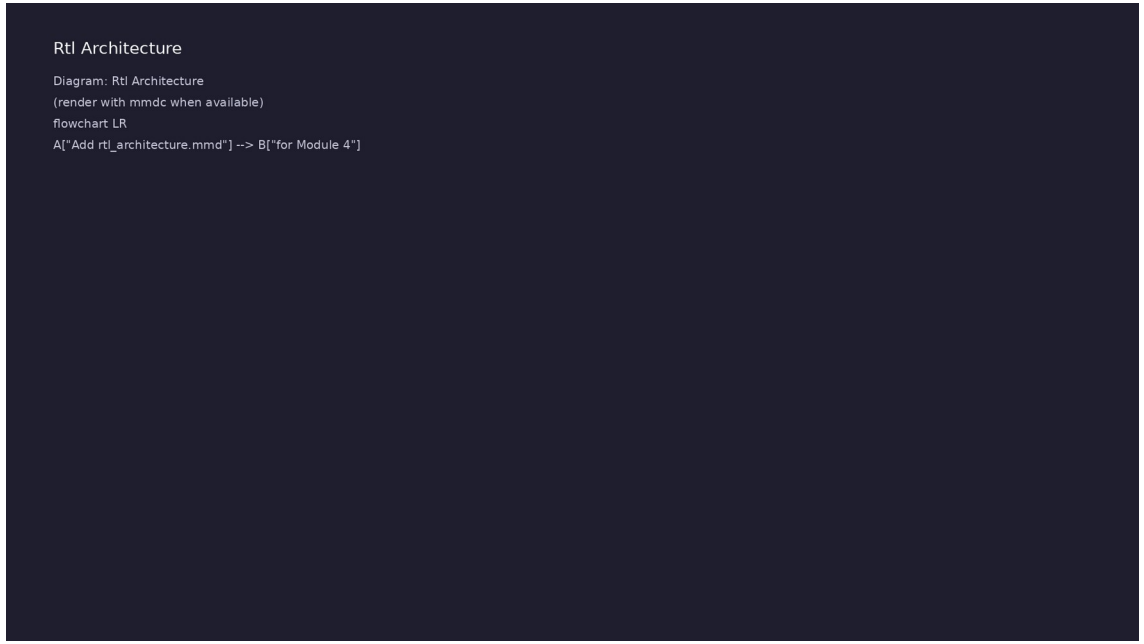
Overview

- This module covers the core UVM components used to build verification environments. You'll learn ho…

Design architecture

1. UVM agent internal architecture (1/3)

- Active agent: sequencer + driver + monitor (generates and observes stimulus)
- Passive agent: monitor only (observation/checking on monitored bus)
- Agent `is_active` config selects build-time component set
- Sequencer-driver TLM: `driver.seq_item_port.connect(sequencer.seq_item_export)`



1. UVM agent internal architecture (2/3)

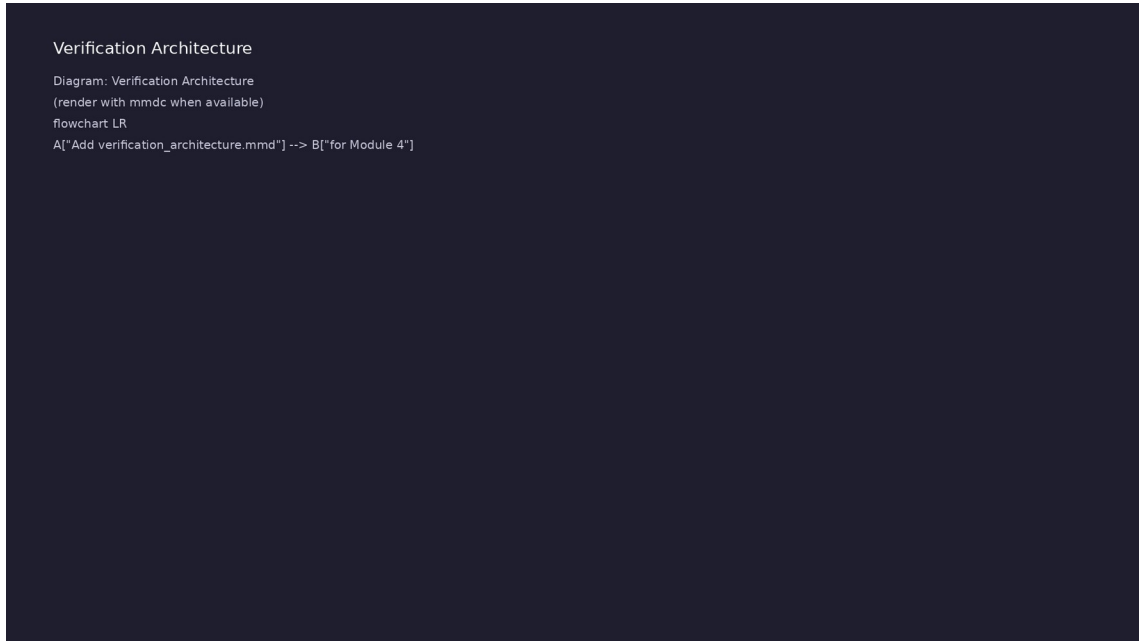
- uvm_sequencer: Arbitrates sequences; exports seq_item_export
- uvm_driver: seq_item_port pulls items; drives virtual interface
- uvm_monitor: Passive observation; broadcasts via analysis port
- uvm_agent: Configures active/passive; builds child components

1. UVM agent internal architecture (3/3)

- uvm_sequence: Produces randomized/constrained transactions

2. Transaction-level data path (1/3)

- TLM ports: `uvm_analysis_port` / `uvm_analysis_export` for monitor → scoreboard
- Transactions implement `do_copy`, `do_compare`, `convert2string` for debug and checking
- Execution sequence: build agent → connect TLM → start sequence in `run_phase`
- 1: Sequence — randomize() transaction fields



2. Transaction-level data path (2/3)

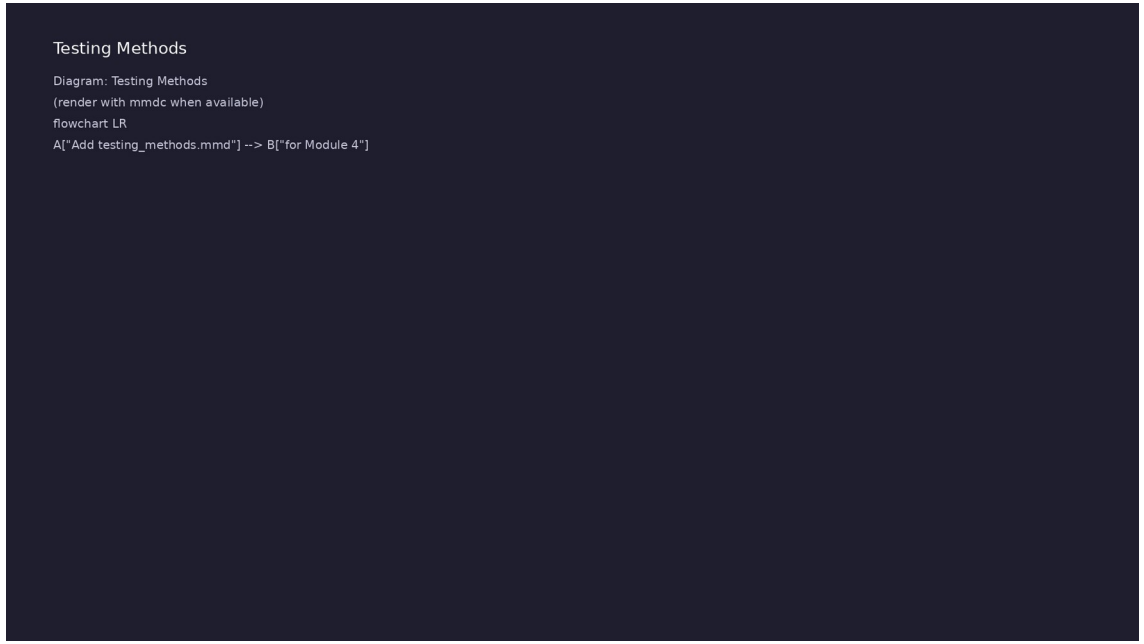
- 2: Sequencer — Sends item to driver via TLM port
- 3: Driver — Drives interface signals from item
- 4: DUT — Responds on bus/interface
- 5: Monitor — Captures pin activity → transaction

2. Transaction-level data path (3/3)

- 6: Scoreboard — Compares expected vs observed
  (Module 4 examples)

3. Module 4 example progression (1/2)

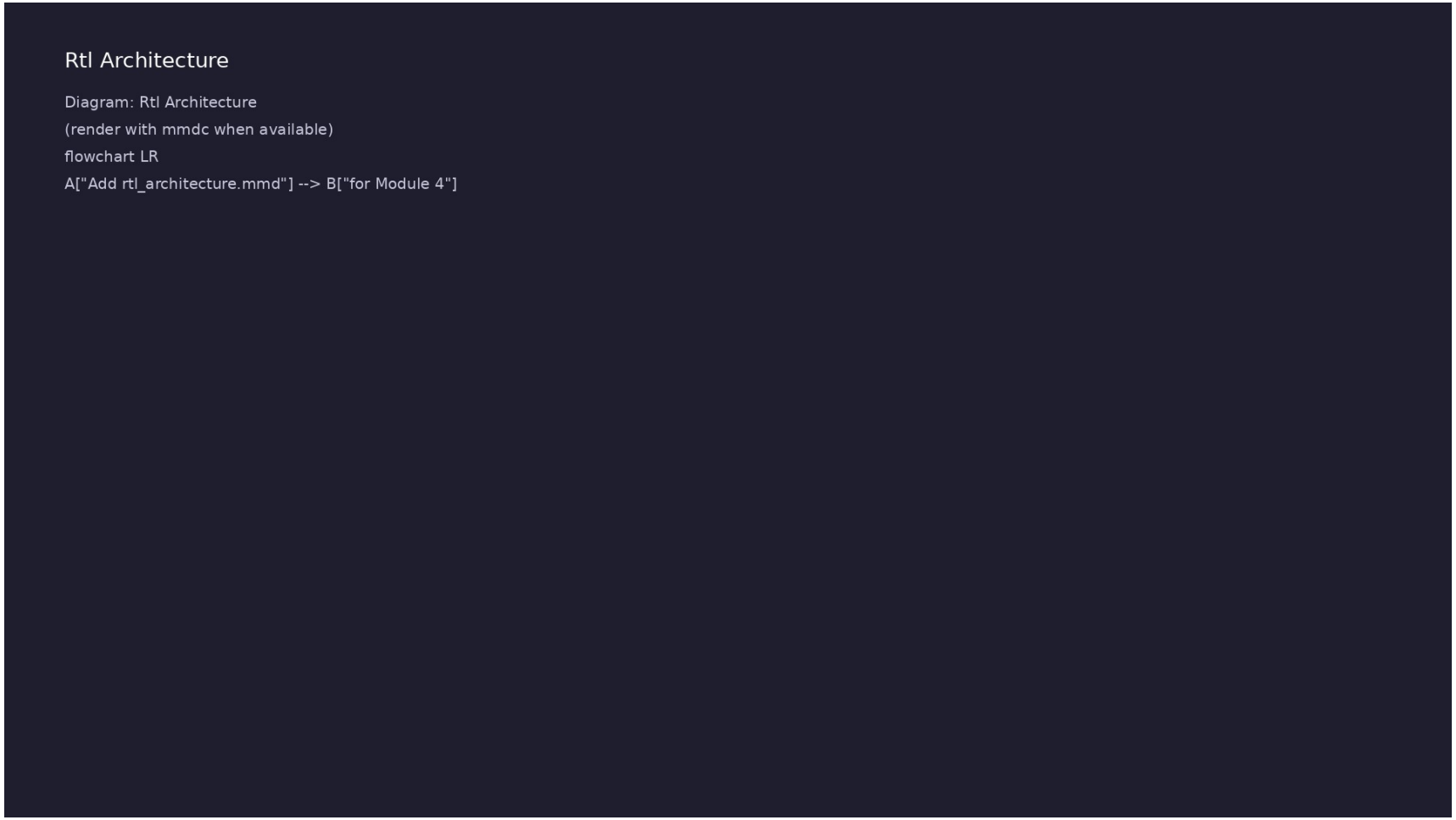
- `examples/transactions/` → data modeling fundamentals
- `examples/drivers/`, `monitors/`, `sequencers/` → individual components
- `examples/tlm/`, `scoreboards/` → communication and checking
- `examples/agents/` → integrated active agent



3. Module 4 example progression (2/2)

- `tests/uvm_tests/test_complete_agent_uvm.sv` — full agent on interface DUT

RTL block diagram (reference)



Module 4: DUT hierarchy and signal flow.

Verification / testbench diagram (reference)

Module 4: stimulus, observation, and checking.

Execution & simulation flow

How the example runs (toolchain)

- Makefile: Verilator + UVM_HOME compile RTL, interface, and test_*.sv
- UVM phases: build → connect → run_test → report (same pattern as Module 2)
- Sequence → driver → DUT → monitor → scoreboard (read SCOREBOARD at end)
- Typical command: make SIM=verilator TEST=test_* (see module4 demo slide)
- Loopback / hooks connect monitor observations to scoreboard checks

Artifact flow (spec → sim) (1)

- 1: Sequence — randomize() transaction fields
- 2: Sequencer — Sends item to driver via TLM port
- 3: Driver — Drives interface signals from item
- 4: DUT — Responds on bus/interface

Artifact flow (spec → sim) (2)

- 5: Monitor — Captures pin activity → transaction
- 6: Scoreboard — Compares expected vs observed

  (Module 4 examples)

Example directory layout

- uvm_sequencer: Arbitrates sequences; exports seq_item_export
- uvm_driver: seq_item_port pulls items; drives virtual interface
- uvm_monitor: Passive observation; broadcasts via analysis port
- uvm_agent: Configures active/passive; builds child components
- uvm_sequence: Produces randomized/constrained transactions

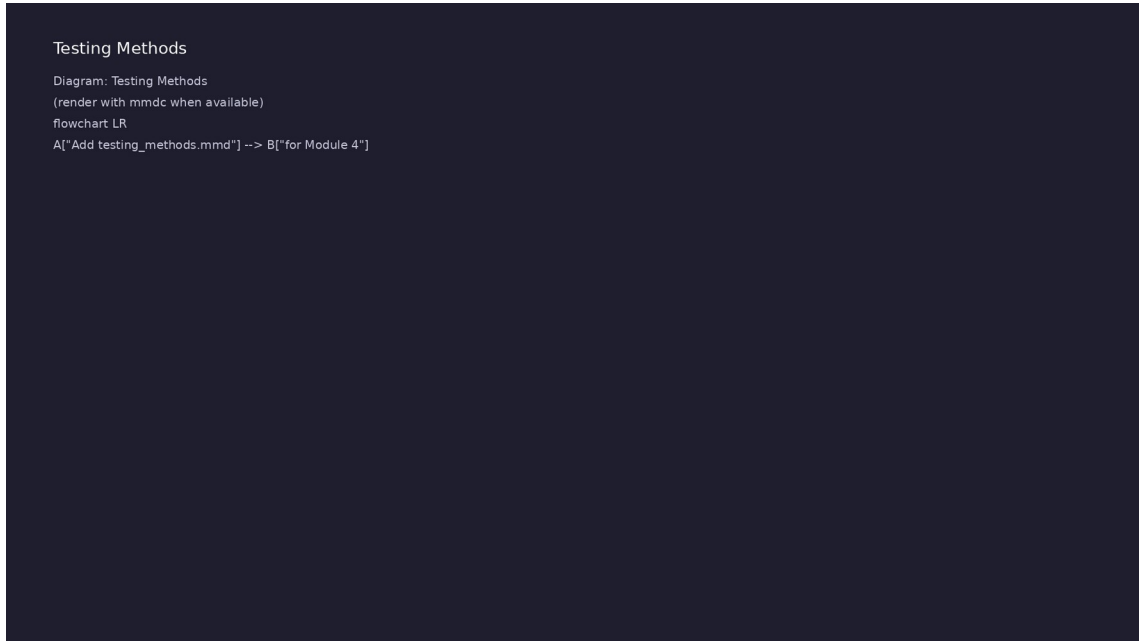
Directed test execution sequence

- build agent
- connect TLM
- start sequence in run_phase

Verification & testing methods

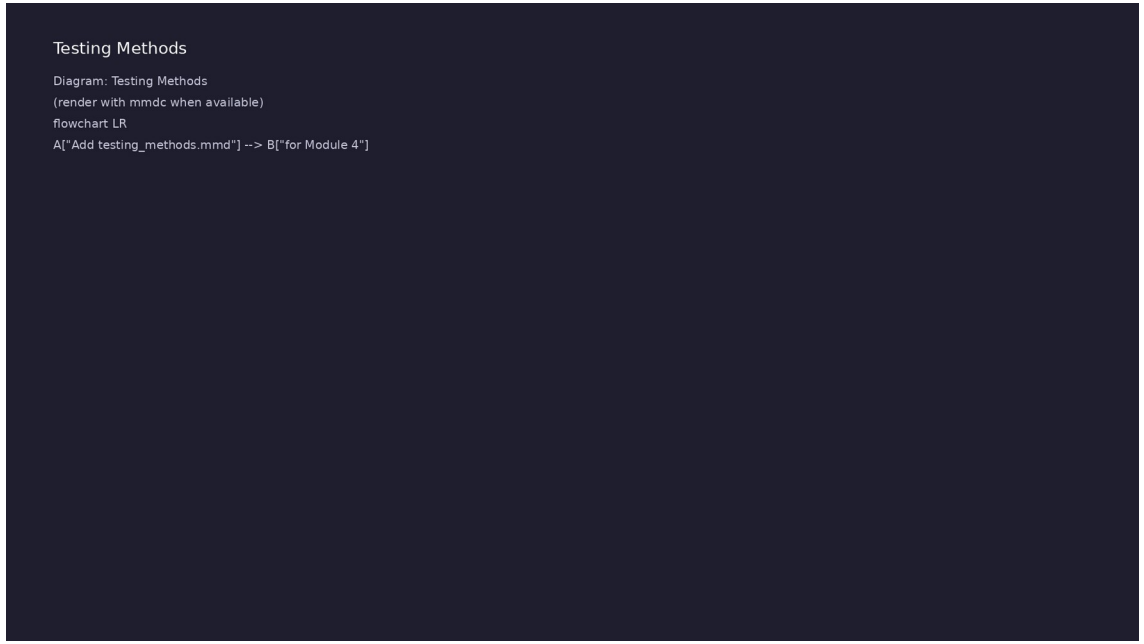
1. Component-level verification strategy

- Verify each component in isolation before agent integration
- Driver: loopback or simple slave model confirms pin wiggles match items
- Monitor: drive known patterns; confirm reconstructed transactions
- Sequencer: run single sequence; count items completed vs requested



2. Agent integration and closure (1/2)

- Connect monitor analysis port to scoreboard export in `connect_phase`
- Use `uvm_config_db` to pass virtual interface to driver and monitor
- Regression: `./scripts/module4.sh --all-examples` then `--uvm-tests`
- Pass criteria: scoreboard match count equals transaction count; no UVM_ERROR



2. Agent integration and closure (2/2)

- Waveform optional — rely on transaction logs and scoreboard summary first

Syllabus topics

Protocol & design details

Detail: Example 4.1: Transactions (`module4/examples/transactions/tra...

- BaseTransaction: Basic transaction with `data` and `address` fields
- ExtendedTransaction: Extended transaction with `control` and `status` fields
- ConstrainedTransaction: Transaction with randomization constraints
- Transaction Operations: Copy, comparison, and string conversion

Detail: Example 4.1: Transactions (`module4/examples/transactions/tra...

- Purpose: Provide human-readable string representation
- Usage: Debugging, logging, reporting
- Key: Override from `uvm_object` base class
- Purpose: Create independent copy of transaction

Detail: Example 4.2: Drivers (`module4/examples/drivers/drivers.sv`)...

- SimpleDriver: Basic driver implementation with transaction reception
- ProtocolDriver: Protocol-aware driver with handshaking
- Driver-Sequencer Communication: Using `seq_item_port` for transaction flow
- Signal Driving: Converting transactions to DUT signals

Detail: Example 4.2: Drivers (`module4/examples/drivers/drivers.sv`)...

- Purpose: Get transaction from sequencer (blocking)
- Usage: Called in driver's `run_phase()` forever loop
- Key: Blocks until sequencer provides transaction
- Purpose: Signal that transaction processing is complete

Detail: Example 4.3: Monitors (`module4/examples/monitors/monitors.sv…

- SimpleMonitor: Basic monitor implementation with signal sampling
- ProtocolMonitor: Protocol-aware monitor with handshake monitoring
- Transaction Creation: Creating transactions from sampled signals
- Analysis Port: Forwarding transactions via analysis port

Detail: Example 4.3: Monitors (`module4/examples/monitors/monitors.sv…

- Purpose: Broadcast transaction to all subscribers
- Usage: Called after creating transaction from sampled signals
- Key: Non-blocking, broadcasts to all connected subscribers
- Purpose: Create analysis port in constructor

Detail: Example 4.4: Sequencers (`module4/examples/sequencers/sequenc…

- SimpleSequence: Basic sequence implementation
- ExtendedSequence: Extended sequence with configurable item count
- Sequence Execution: Starting sequences on sequencers
- Sequence Item Generation: Creating and sending sequence items

Detail: Example 4.4: Sequencers (`module4/examples/sequencers/sequenc…

- Purpose: Generate and send sequence items
- Usage: Main sequence logic
- Key: Called when sequence is started
- Purpose: Request sequence item from sequencer

Detail: Example 4.5: TLM (`module4/examples/tlm/tlm.sv`) (1)

- TlmProducer: Uses `uvm_blocking_put_port` to produce transactions
- TlmConsumer: Uses `uvm_blocking_get_export` to consume transactions
- AnalysisProducer: Uses `uvm_analysis_port` to broadcast transactions
- AnalysisSubscriber: Uses `uvm_subscriber` to receive transactions

Detail: Example 4.5: TLM (`module4/examples/tlm/tlm.sv`) (2)

- Purpose: Send transaction to consumer (blocking)
- Usage: Producer sends transactions
- Key: Blocks until consumer receives
- Purpose: Receive transaction from producer (blocking)

Detail: Example 4.6: Scoreboards (`module4/examples/scoreboards/score...

- SimpleScoreboard: Basic scoreboard with transaction comparison
- Transaction Storage: Storing expected and actual transactions
- Comparison Logic: Comparing transactions for verification
- Result Reporting: Reporting pass/fail counts

Detail: Example 4.6: Scoreboards (`module4/examples/scoreboards/score...

- Purpose: Receive transactions from monitor
- Usage: Called automatically when monitor writes to analysis port
- Key: Compares actual vs expected results
- Purpose: Store expected transaction for later comparison

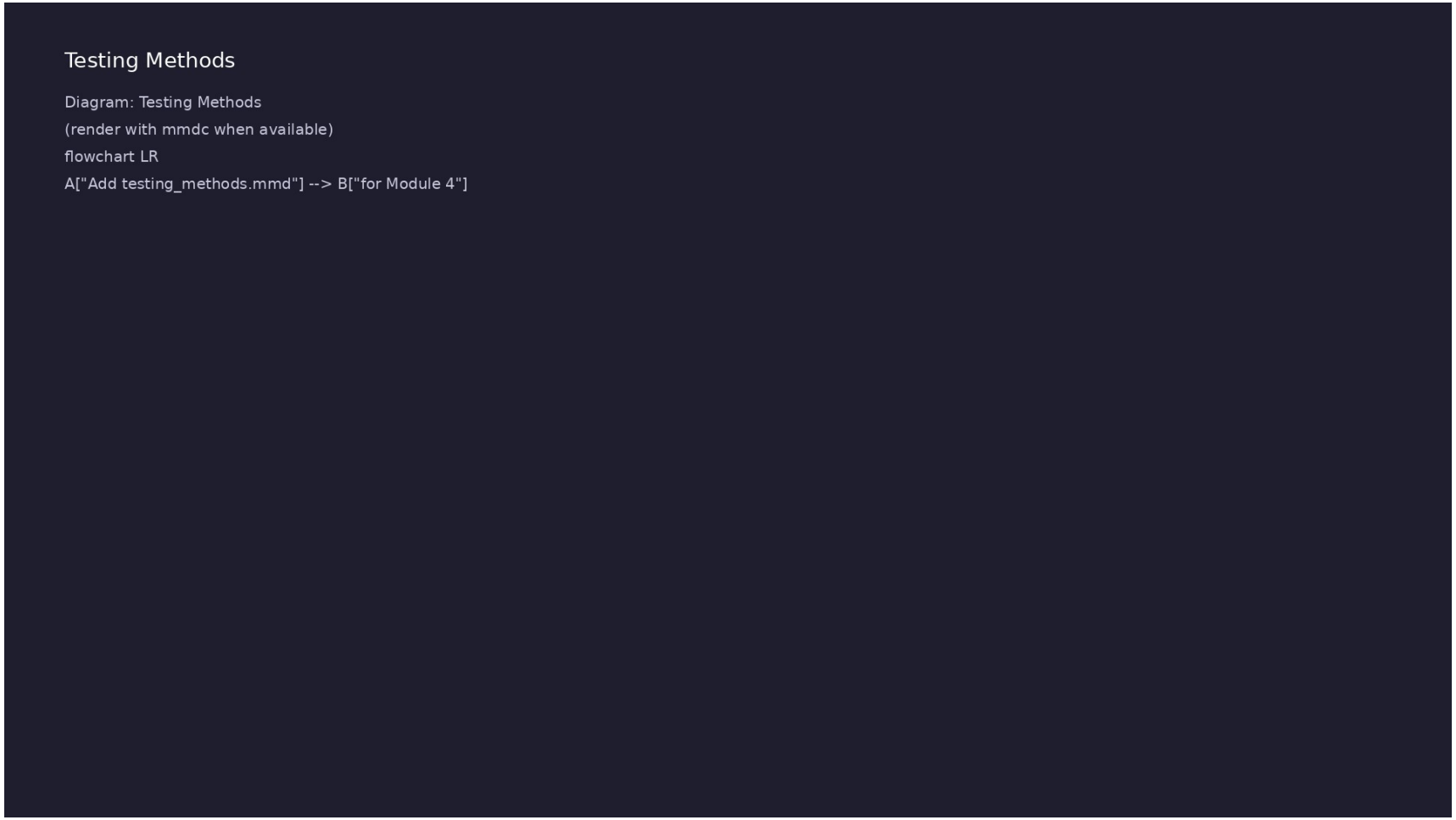
Detail: Example 4.7: Complete Agents (`module4/examples/agents/agents...

- CompleteAgent: Contains driver, monitor, and sequencer
- Agent Configuration: Configurable as active or passive
- Component Integration: Integrating all agent components
- Agent Testing: Testing complete agent functionality

Detail: Example 4.7: Complete Agents (`module4/examples/agents/agents...

- Purpose: Create agent components based on configuration
- Key: Active agent has driver+sequencer, passive has monitor only
- Monitor: Always created (both active and passive modes)
- Driver/Sequencer: Only created in active mode

Verification flow (reference)



Stimulus, DUT response, and check points for this module.

1. UVM Agent Architecture (1/9)

- Agent Overview and Structure
- Agent purpose and responsibilities
- Agent components (driver, monitor, sequencer)
- Agent configuration
- Active vs passive agents
- Agent Components

1. UVM Agent Architecture (2/9)

- Driver for stimulus generation
- Monitor for observation
- Sequencer for sequence management
- Agent integration
- BaseTransaction: Basic transaction with `data` and `address` fields
- ExtendedTransaction: Extended transaction with `control` and `status` fields

1. UVM Agent Architecture (3/9)

- ConstrainedTransaction: Transaction with randomization constraints
- Transaction Operations: Copy, comparison, and string conversion
- Purpose: Provide human-readable string representation
- Usage: Debugging, logging, reporting
- Key: Override from `uvm_object` base class
- Purpose: Create independent copy of transaction

1. UVM Agent Architecture (4/9)

- Usage: `txn2.copy(txn1)` - creates deep copy
- Key: Must call `super.do_copy(rhs)` first, then copy all fields
- Purpose: Compare two transactions for equality
- Usage: `if (txn1.compare(txn2))` - returns 1 if equal
- Key: Returns 1 if all fields match, 0 otherwise
- Purpose: Generate random values satisfying constraints

1. UVM Agent Architecture (5/9)

- Usage: `txn.randomize()` - returns 1 if successful
- Key: Constraints ensure protocol compliance
- Purpose: Post-process after randomization
- Usage: Calculate derived fields, validate constraints
- Key: Called automatically after `randomize()` succeeds
- Example: Calculate expected results, checksums, derived values

1. UVM Agent Architecture (6/9)

- Purpose: Type-safe conversion from `uvm_object` to specific type
- Usage: Required in `do_copy()` and `do_compare()`
- Key: Always check return value, handle type mismatch
- Purpose: Extend base transaction with additional fields
- Key: Must call `super` methods to handle base class fields
- Transaction class design extending `uvm_sequence_item`

1. UVM Agent Architecture (7/9)

- Transaction fields and data members
- Transaction copy and comparison operations
- Extended transactions with inheritance
- Constrained random transactions
- `convert2string()` for debugging and logging
- `do_copy()` for deep copying transactions

1. UVM Agent Architecture (8/9)

- `do_compare()` for transaction comparison
- `randomize()` for constrained random generation
- Constraints ensure protocol compliance
- Transaction creation and initialization
- Transaction copy operations
- Transaction comparison operations

1. UVM Agent Architecture (9/9)

- Extended transaction inheritance
- Constrained random transaction generation

2. UVM Driver Implementation (1/8)

- Driver Purpose and Responsibilities
- Stimulus generation
- Protocol implementation
- Signal driving
- Transaction-to-signal conversion
- Driver Implementation

2. UVM Driver Implementation (2/8)

- Driver class structure
- Sequencer communication
- Transaction reception
- Signal driving patterns
- SimpleDriver: Basic driver implementation with transaction reception
- ProtocolDriver: Protocol-aware driver with handshaking

2. UVM Driver Implementation (3/8)

- Driver-Sequencer Communication: Using `seq_item_port` for transaction flow
- Signal Driving: Converting transactions to DUT signals
- Purpose: Get transaction from sequencer (blocking)
- Usage: Called in driver's `run_phase()` forever loop
- Key: Blocks until sequencer provides transaction
- Purpose: Signal that transaction processing is complete

2. UVM Driver Implementation (4/8)

- Usage: Called after driving transaction to DUT
- Key: Allows sequencer to provide next transaction
- Purpose: Convert transaction to DUT signals
- Usage: Called after `get_next_item()`
- Key: Implements protocol-specific driving logic
- Purpose: Implement protocol-specific handshaking

2. UVM Driver Implementation (5/8)

- Key: Handles request/grant, timing, protocol compliance
- Purpose: Connect driver to sequencer
- Usage: In agent's `connect_phase()`
- Key: Establishes TLM communication channel
- Driver class structure extending `uvm_driver`
- Transaction reception from sequencer via `seq_item_port`

2. UVM Driver Implementation (6/8)

- Signal driving to DUT
- Driver-sequencer communication
- Protocol-specific driving patterns
- `get_next_item()` - Get transaction from sequencer (blocking)
- `item_done()` - Signal transaction completion
- `drive_transaction()` - Convert transaction to signals

2. UVM Driver Implementation (7/8)

- TLM connection - `seq_item_port.connect(sequencer.seq_item_export)`
- Forever loop - Driver continuously processes transactions
- Driver creation and initialization
- Transaction reception from sequencer
- Signal driving to DUT
- Protocol-specific driving patterns

2. UVM Driver Implementation (8/8)

- Driver-sequencer communication

3. UVM Monitor Implementation (1/7)

- Monitor Purpose and Responsibilities
- DUT observation
- Signal sampling
- Transaction creation
- Analysis port usage
- Monitor Implementation

3. UVM Monitor Implementation (2/7)

- Monitor class structure
- Signal sampling
- Transaction creation
- Analysis port forwarding
- SimpleMonitor: Basic monitor implementation with signal sampling
- ProtocolMonitor: Protocol-aware monitor with handshake monitoring

3. UVM Monitor Implementation (3/7)

- Transaction Creation: Creating transactions from sampled signals
- Analysis Port: Forwarding transactions via analysis port
- Purpose: Broadcast transaction to all subscribers
- Usage: Called after creating transaction from sampled signals
- Key: Non-blocking, broadcasts to all connected subscribers
- Purpose: Create analysis port in constructor

3. UVM Monitor Implementation (4/7)

- Key: Analysis ports created in constructor, not build_phase
- Purpose: Sample DUT signals and create transactions
- Key: Monitor is passive (doesn't drive DUT)
- Purpose: Connect monitor to subscribers (scoreboard, coverage, etc.)
- Key: One monitor can connect to multiple subscribers
- Purpose: Receive transactions from analysis port

3. UVM Monitor Implementation (5/7)

- Usage: Scoreboard, coverage collector, logger
- Key: Automatically called when monitor writes to analysis port
- Monitor class structure extending `uvm_monitor`
- Signal sampling from DUT
- Transaction creation from sampled signals
- Analysis port usage

3. UVM Monitor Implementation (6/7)

- Protocol-specific monitoring
- `ap.write(txn)` - Broadcast transaction to subscribers
- Analysis port creation - In constructor, not build_phase
- Signal sampling - Wait for protocol events, sample signals
- Transaction creation - Create from sampled signals
- Subscriber pattern - Components receive via `write()` method

3. UVM Monitor Implementation (7/7)

- Monitor creation and initialization
- Signal sampling from DUT
- Transaction creation from signals
- Analysis port forwarding
- Protocol-specific monitoring

4. UVM Sequencer and Sequences (1/8)

- Sequencer Implementation
- Sequencer purpose
- Sequence item management
- Sequence execution
- Sequencer-driver communication
- Sequence Implementation

4. UVM Sequencer and Sequences (2/8)

- Sequence items (`uvm_sequence_item`)
- Sequences (`uvm_sequence`)
- Sequence execution
- Sequence libraries
- SimpleSequence: Basic sequence implementation
- ExtendedSequence: Extended sequence with
  configurable item count

4. UVM Sequencer and Sequences (3/8)

- Sequence Execution: Starting sequences on sequencers
- Sequence Item Generation: Creating and sending sequence items
- Purpose: Generate and send sequence items
- Usage: Main sequence logic
- Key: Called when sequence is started
- Purpose: Request sequence item from sequencer

4. UVM Sequencer and Sequences (4/8)

- Usage: Called before setting item fields
- Key: Blocking call, waits for sequencer availability
- Purpose: Send sequence item to driver
- Usage: Called after setting item fields
- Key: Item flows: Sequence → Sequencer → Driver
- Purpose: Start sequence execution on sequencer

4. UVM Sequencer and Sequences (5/8)

- Usage: In test's `run_phase()`
- Key: Sequence executes `body()` task
- Purpose: Generate random test vectors
- Key: Constraints ensure valid values
- Purpose: Configurable sequence with parameters
- Key: Can be configured via ConfigDB or constructor

4. UVM Sequencer and Sequences (6/8)

- Sequencer implementation
- Sequence items (`uvm_sequence_item`)
- Sequences (`uvm_sequence`)
- Sequence execution
- Sequence libraries
- `body()` - Main sequence logic (generates items)

4. UVM Sequencer and Sequences (7/8)

- `start_item(item)` - Request item from sequencer
- `finish_item(item)` - Send item to driver
- `seq.start(sequencer)` - Start sequence execution
- Item flow: Sequence → Sequencer → Driver
- Sequencer creation and initialization
- Sequence item generation

4. UVM Sequencer and Sequences (8/8)

- Sequence execution
- Sequencer-driver communication
- Sequence libraries

5. TLM (Transaction-Level Modeling) (1/8)

- TLM Interfaces
- Put/get interfaces
- Peek interfaces
- Transport interfaces
- Analysis interfaces
- TLM Components

5. TLM (Transaction-Level Modeling) (2/8)

- TLM ports and exports
- TLM FIFOs
- Analysis ports and exports
- TLM connections
- TlmProducer: Uses `uvm_blocking_put_port` to produce transactions
- TlmConsumer: Uses `uvm_blocking_get_export` to consume transactions

5. TLM (Transaction-Level Modeling) (3/8)

- AnalysisProducer: Uses `uvm_analysis_port` to broadcast transactions
- AnalysisSubscriber: Uses `uvm_subscriber` to receive transactions
- Purpose: Send transaction to consumer (blocking)
- Usage: Producer sends transactions
- Key: Blocks until consumer receives
- Purpose: Receive transaction from producer (blocking)

5. TLM (Transaction-Level Modeling) (4/8)

- Usage: Consumer receives transactions
- Key: Blocks until producer sends
- Purpose: Buffer between producer and consumer
- Usage: Decouples producer and consumer timing
- Key: FIFO stores transactions temporarily
- Purpose: Broadcast transaction to multiple subscribers

5. TLM (Transaction-Level Modeling) (5/8)

- Usage: One-to-many communication
- Key: Non-blocking, broadcasts to all subscribers
- Purpose: Receive transactions from analysis port
- Usage: Scoreboard, coverage, logger
- Key: Automatically called when producer writes
- TLM interfaces (put, get, peek, transport)

5. TLM (Transaction-Level Modeling) (6/8)

- TLM ports and exports
- TLM FIFOs
- Analysis ports and exports
- TLM connections
- `put_port.put(txn)` - Send transaction (blocking)
- `get_export.get(txn)` - Receive transaction (blocking)

5. TLM (Transaction-Level Modeling) (7/8)

- `ap.write(txn)` - Broadcast transaction (non-blocking)
- `write(t)` - Receive in subscriber (automatic)
- TLM FIFO - Decouples producer and consumer
- Analysis port - One-to-many communication
- TLM port/export creation
- TLM connection establishment

5. TLM (Transaction-Level Modeling) (8/8)

- Transaction flow via TLM
- Analysis port broadcasting
- TLM FIFO usage

6. Scoreboards (1/8)

- Scoreboard Purpose
- Result checking
- Transaction comparison
- Pass/fail tracking
- Verification closure
- Scoreboard Implementation

6. Scoreboards (2/8)

- Scoreboard structure
- Transaction storage
- Comparison logic
- Result reporting
- SimpleScoreboard: Basic scoreboard with transaction comparison
- Transaction Storage: Storing expected and actual transactions

6. Scoreboards (3/8)

- Comparison Logic: Comparing transactions for verification
- Result Reporting: Reporting pass/fail counts
- Purpose: Receive transactions from monitor
- Usage: Called automatically when monitor writes to analysis port
- Key: Compares actual vs expected results
- Purpose: Store expected transaction for later comparison

6. Scoreboards (4/8)

- Usage: Called before actual transaction arrives
- Key: Queue stores expected transactions in order
- Purpose: Manage expected transaction queue
- Key: FIFO queue ensures order matching
- Purpose: Compare actual vs expected results
- Key: Reports pass/fail, tracks counts

6. Scoreboards (5/8)

- Purpose: Report final test results
- Usage: Called in cleanup phase
- Key: Provides test pass/fail summary
- Purpose: Connect monitor to scoreboard
- Key: Monitor writes → Scoreboard receives
- Scoreboard purpose and structure

6. Scoreboards (6/8)

- Transaction comparison
- Result checking
- Pass/fail tracking
- `write(txn)` - Receive transaction from monitor
- `add_expected(txn)` - Store expected transaction
- Queue operations - `push_back()`, `pop_front()`, `size()`

6. Scoreboards (7/8)

- Comparison logic - Compare actual vs expected
- Report phase - Final test results
- Analysis export - `uvm_analysis_imp` receives transactions
- Scoreboard creation and initialization
- Transaction storage
- Transaction comparison

6. Scoreboards (8/8)

- Pass/fail tracking
- Result reporting

7. Complete Agent Example (1/6)

- Building Complete Agents
- Agent structure
- Component integration
- Agent configuration
- Active vs passive agents
- CompleteAgent: Contains driver, monitor, and sequencer

7. Complete Agent Example (2/6)

- Agent Configuration: Configurable as active or passive
- Component Integration: Integrating all agent components
- Agent Testing: Testing complete agent functionality
- Purpose: Create agent components based on configuration
- Key: Active agent has driver+sequencer, passive has monitor only
- Monitor: Always created (both active and passive modes)

7. Complete Agent Example (3/6)

- Driver/Sequencer: Only created in active mode
- Purpose: Connect agent components
- Key: Driver-sequencer connection only for active agents
- Note: Monitor analysis port connected in environment's connect_phase
- Purpose: Configure agent as active or passive
- Usage: Set before creating agent

7. Complete Agent Example (4/6)

- Key: Active = driver+sequencer+monitor, Passive = monitor only
- Timing: Must be set in test's build_phase before agent creation
- Purpose: Use agent based on mode
- Key: Check `is_active` before using driver/sequencer
- Agent structure
- Component integration

7. Complete Agent Example (5/6)

- Agent configuration
- Active vs passive agents
- Active agent - Driver + Sequencer + Monitor
- Passive agent - Monitor only
- Configuration - Via ConfigDB before creation
- Component creation - Based on active/passive mode

7. Complete Agent Example (6/6)

- Connection - Driver-sequencer only for active agents
- Complete agent creation
- Component integration
- Agent configuration
- Active/passive agent modes
- Agent functionality

Run: Example 4.1: Transactions

```
./scripts/module4.sh --transactions
cd module4/examples/transactions
verilator -sv --cc --exe --timing --trace \
-I$UVM_HOME/src +incdir+$UVM_HOME/src +define+UVM_NO_DPI \
--binary $UVM_HOME/src/uvm_pkg.sv transactions.sv transactions.cpp \
--top-module transactions
make -C obj_dir -f Vtransactions.mk
./obj_dir/Vtransactions
```

Run: Example 4.2: Drivers

```
./scripts/module4.sh --drivers
```

Run: Example 4.3: Monitors

```
./scripts/module4.sh --monitors
```

Run: Example 4.4: Sequencers

```
./scripts/module4.sh --sequencers
```

Run: Example 4.5: TLM

```
./scripts/module4.sh --tlm
```

Run: Example 4.6: Scoreboards

```
./scripts/module4.sh --scoreboards
```

Run: Example 4.7: Complete Agents

```
./scripts/module4.sh --agents
```

Hands-on examples

Module 4 self-check

```
$ ./scripts/module4.sh --help
```

Terminal: module4_check

```
Usage: ./scripts/module4.sh [OPTIONS]

Module 4: UVM Components
This script runs examples and tests for Module 4.

OPTIONS:
    UVM Examples:
        --transactions      Run transaction examples
        --drivers           Run driver examples
        --monitors          Run monitor examples
        --sequencers        Run sequencer examples
        --tlm               Run TLM examples
        --scoreboards       Run scoreboard examples
        --agents            Run agent examples
        --all-examples      Run all UVM examples (default)
        --skip-examples     Skip all UVM examples

    Tests:
        --uvm-tests         Run UVM testbenches
        --all-tests         Run all tests

    Environment:
        --sim SIMULATOR     Simulator to use (default: verilator)
        --uvm-home DIR      UVM library directory (default: $UVM_HOME)
        --jobs N            Number of parallel build jobs (default: 8)
        --no-clean          Skip cleaning before build (faster rebuilds)

    Other:
        --help, -h          Show this help message

EXAMPLES:
    # Run all UVM examples (with parallel builds)
    ./scripts/module4.sh

    # Run only transactions
    ./scripts/module4.sh --transactions
```

Exercise scaffold

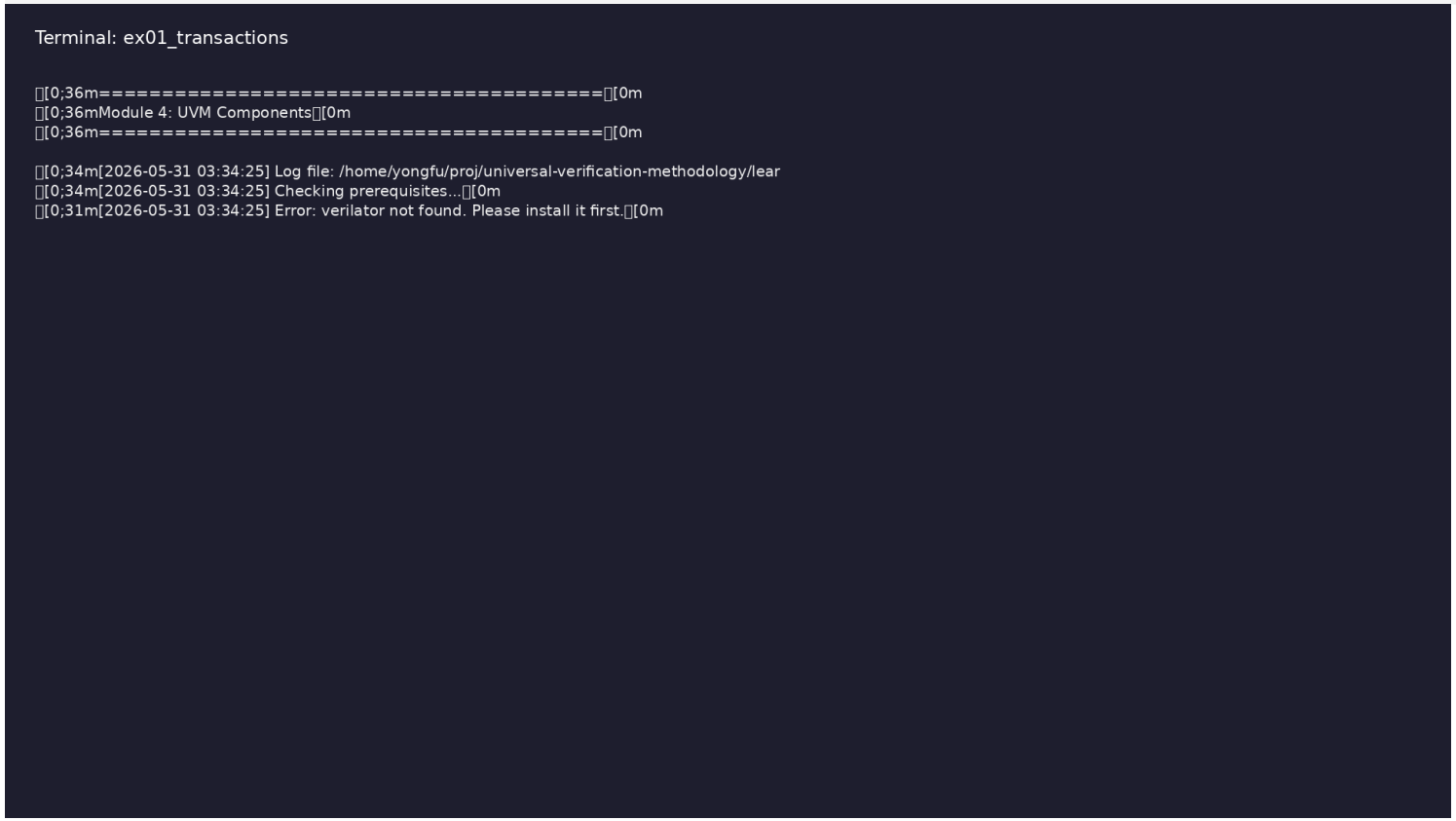
```
./scripts/module4.sh --scaffold
```

Example 1: Transactions

- BaseTransaction: Basic transaction with `data` and `address` fields
- ExtendedTransaction: Extended transaction with `control` and `status` fields
- ConstrainedTransaction: Transaction with randomization constraints
- Transaction Operations: Copy, comparison, and string conversion

Demo: Transactions

```
$ ./scripts/module4.sh --transactions
```

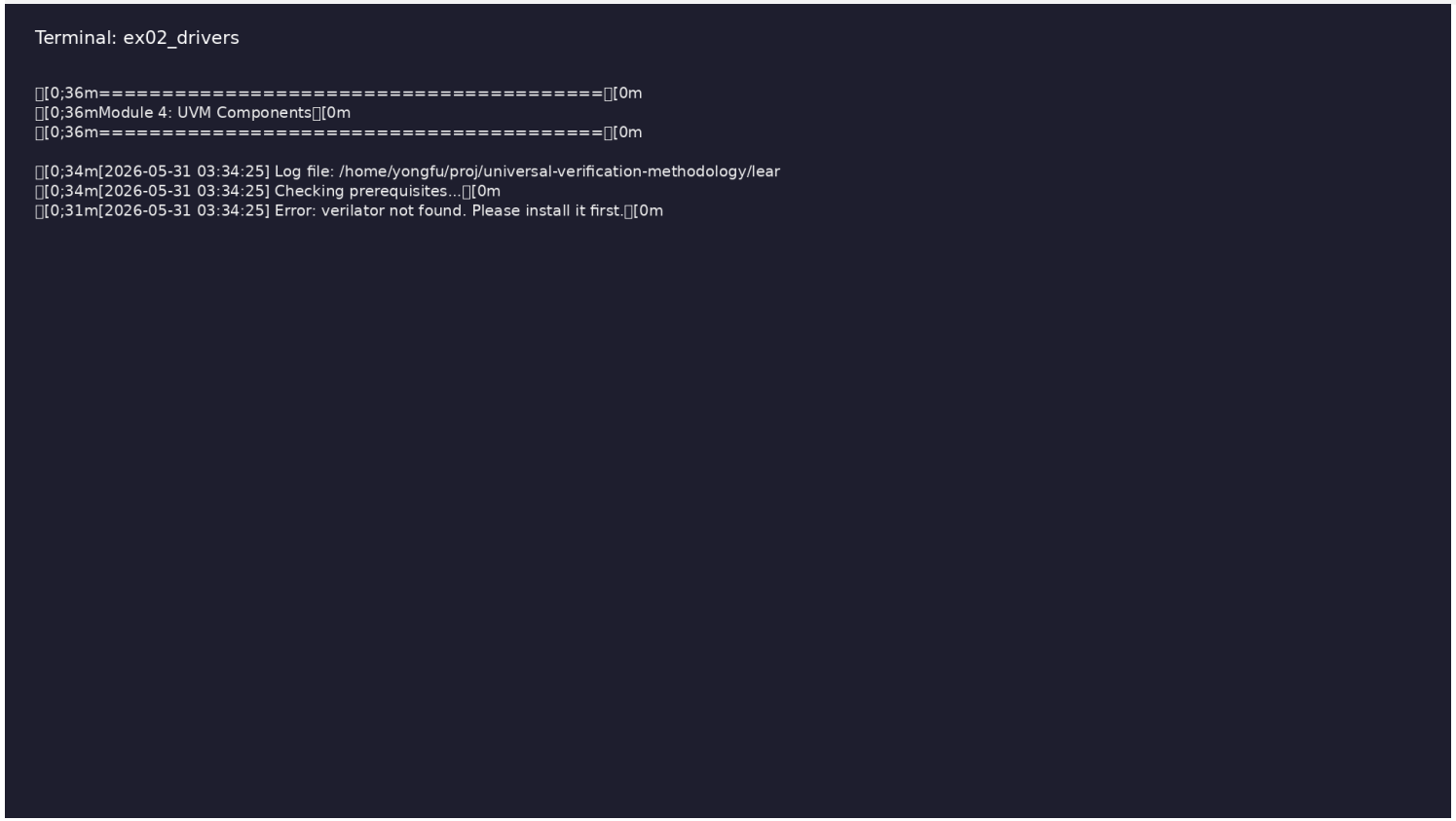


Example 2: Drivers

- SimpleDriver: Basic driver implementation with transaction reception
- ProtocolDriver: Protocol-aware driver with handshaking
- Driver-Sequencer Communication: Using `seq_item_port` for transaction flow
- Signal Driving: Converting transactions to DUT signals

Demo: Drivers

```
$ ./scripts/module4.sh --drivers
```

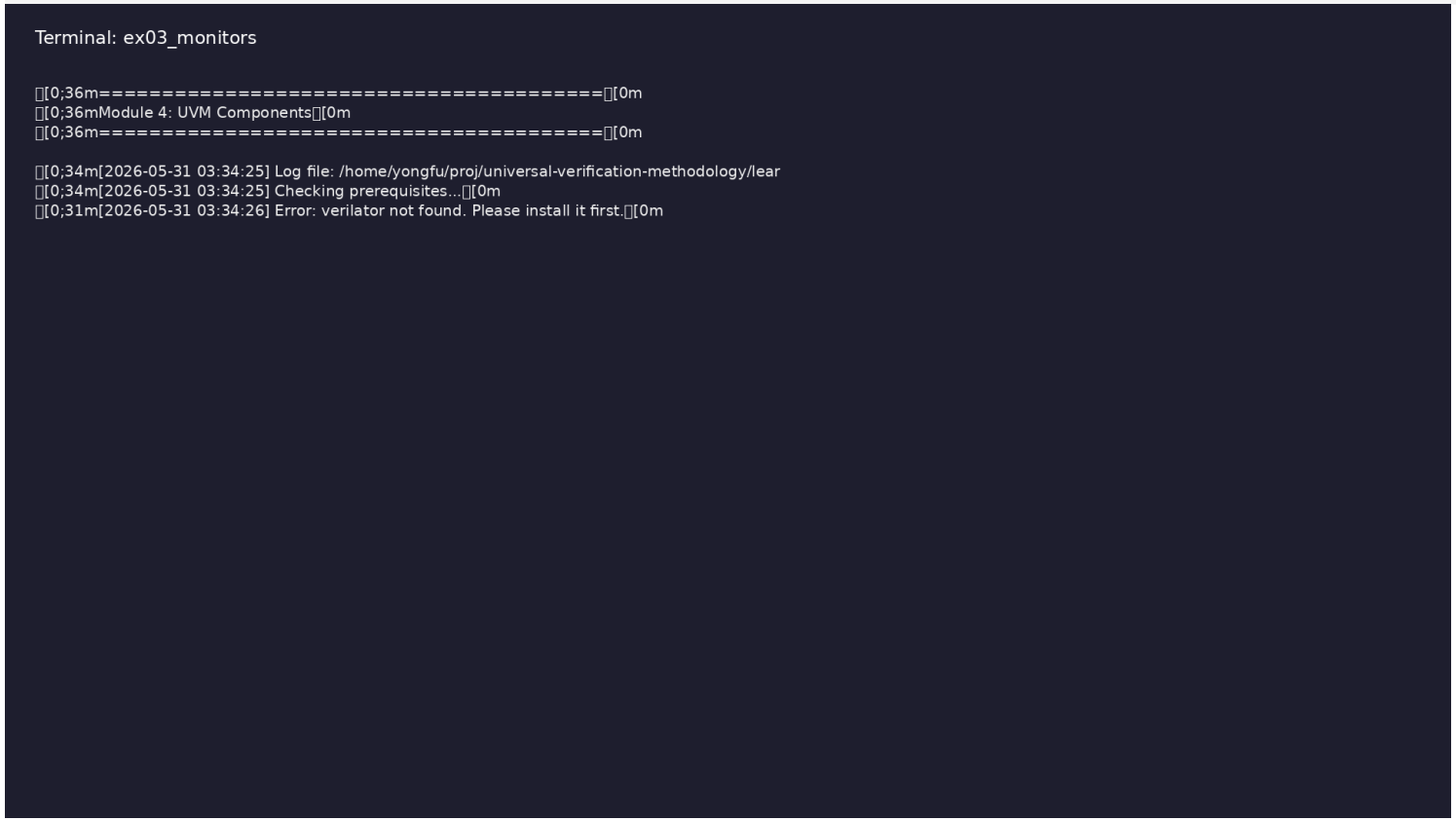


Example 3: Monitors

- SimpleMonitor: Basic monitor implementation with signal sampling
- ProtocolMonitor: Protocol-aware monitor with handshake monitoring
- Transaction Creation: Creating transactions from sampled signals
- Analysis Port: Forwarding transactions via analysis port

Demo: Monitors

```
$ ./scripts/module4.sh --monitors
```

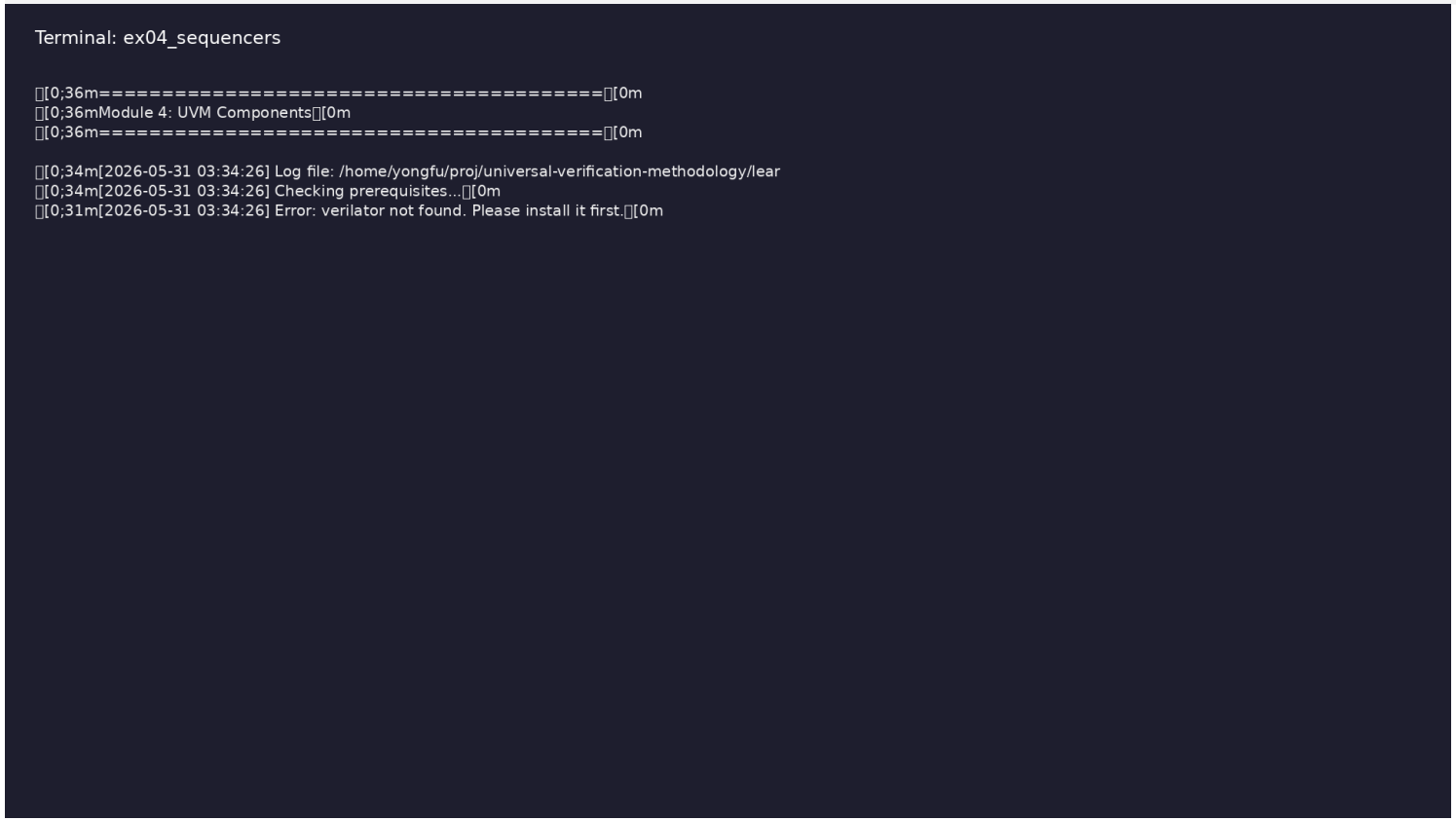


Example 4: Sequencers

- SimpleSequence: Basic sequence implementation
- ExtendedSequence: Extended sequence with configurable item count
- Sequence Execution: Starting sequences on sequencers
- Sequence Item Generation: Creating and sending sequence items

Demo: Sequencers

```
$ ./scripts/module4.sh --sequencers
```

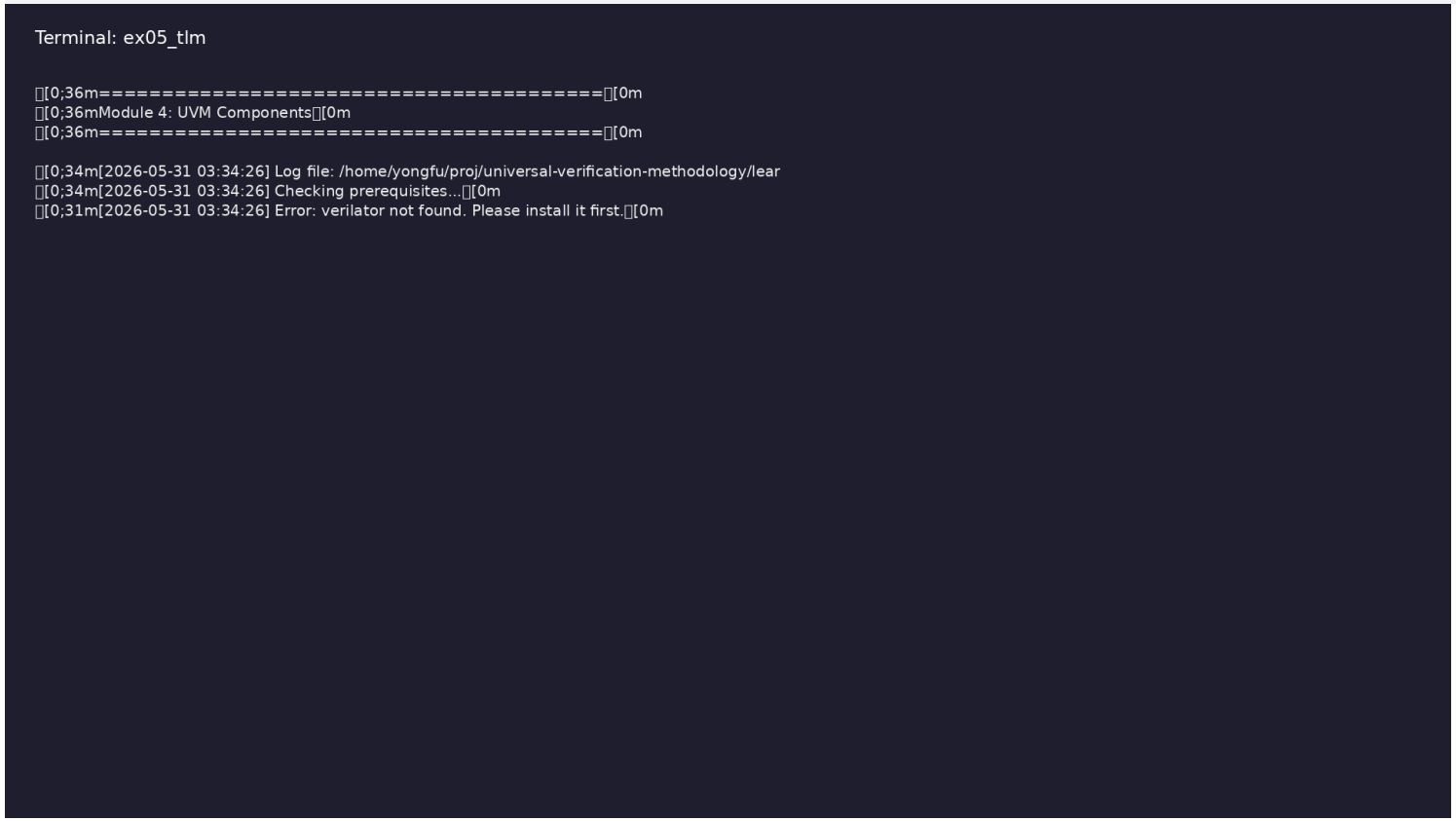


Example 5: TLM

- TlmProducer: Uses `uvm_blocking_put_port` to produce transactions
- TlmConsumer: Uses `uvm_blocking_get_export` to consume transactions
- AnalysisProducer: Uses `uvm_analysis_port` to broadcast transactions
- AnalysisSubscriber: Uses `uvm_subscriber` to receive transactions

Demo: TLM

```
$ ./scripts/module4.sh --tlm
```

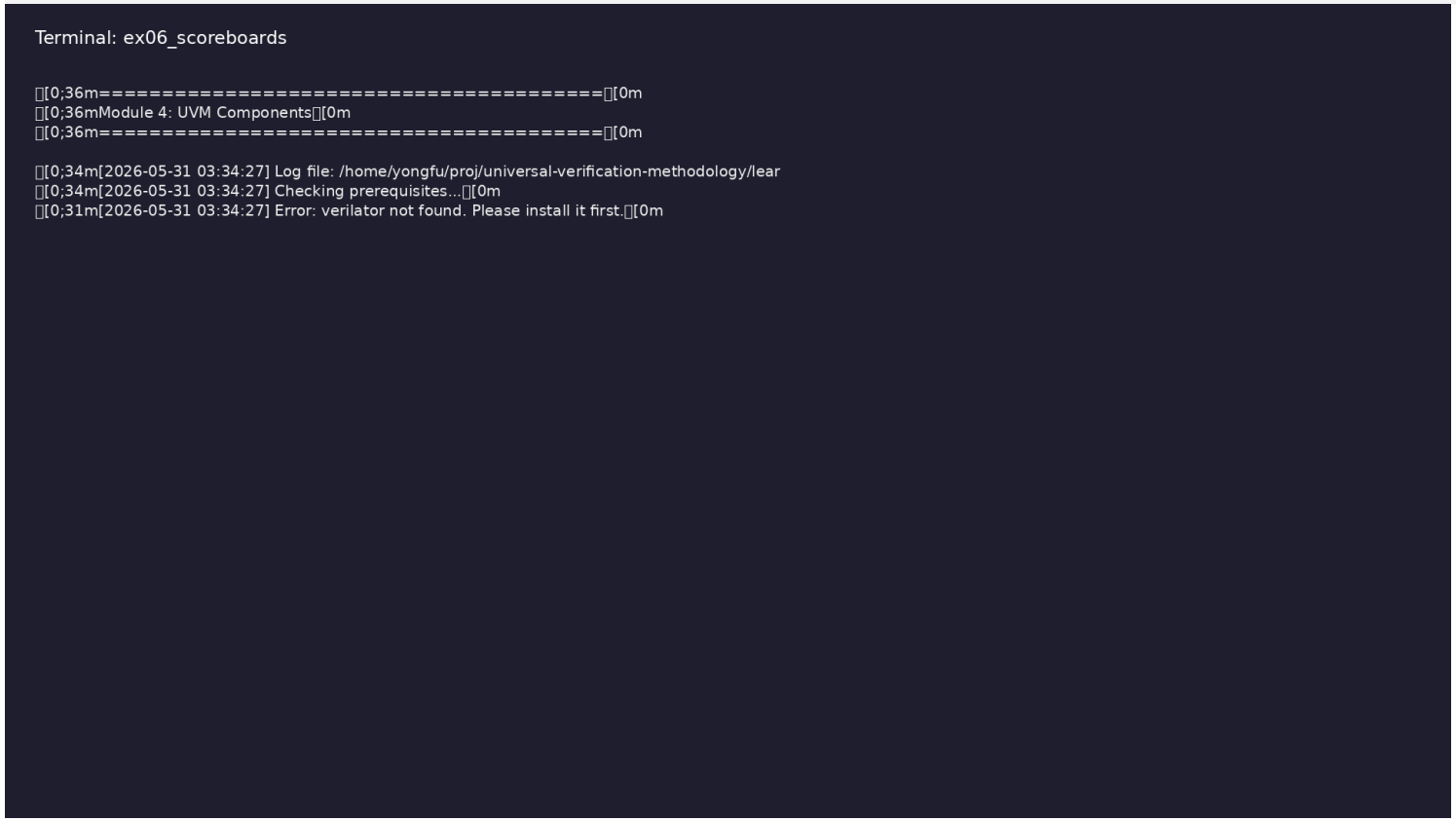


Example 6: Scoreboards

- SimpleScoreboard: Basic scoreboard with transaction comparison
- Transaction Storage: Storing expected and actual transactions
- Comparison Logic: Comparing transactions for verification
- Result Reporting: Reporting pass/fail counts

Demo: Scoreboards

```
$ ./scripts/module4.sh --scoreboards
```

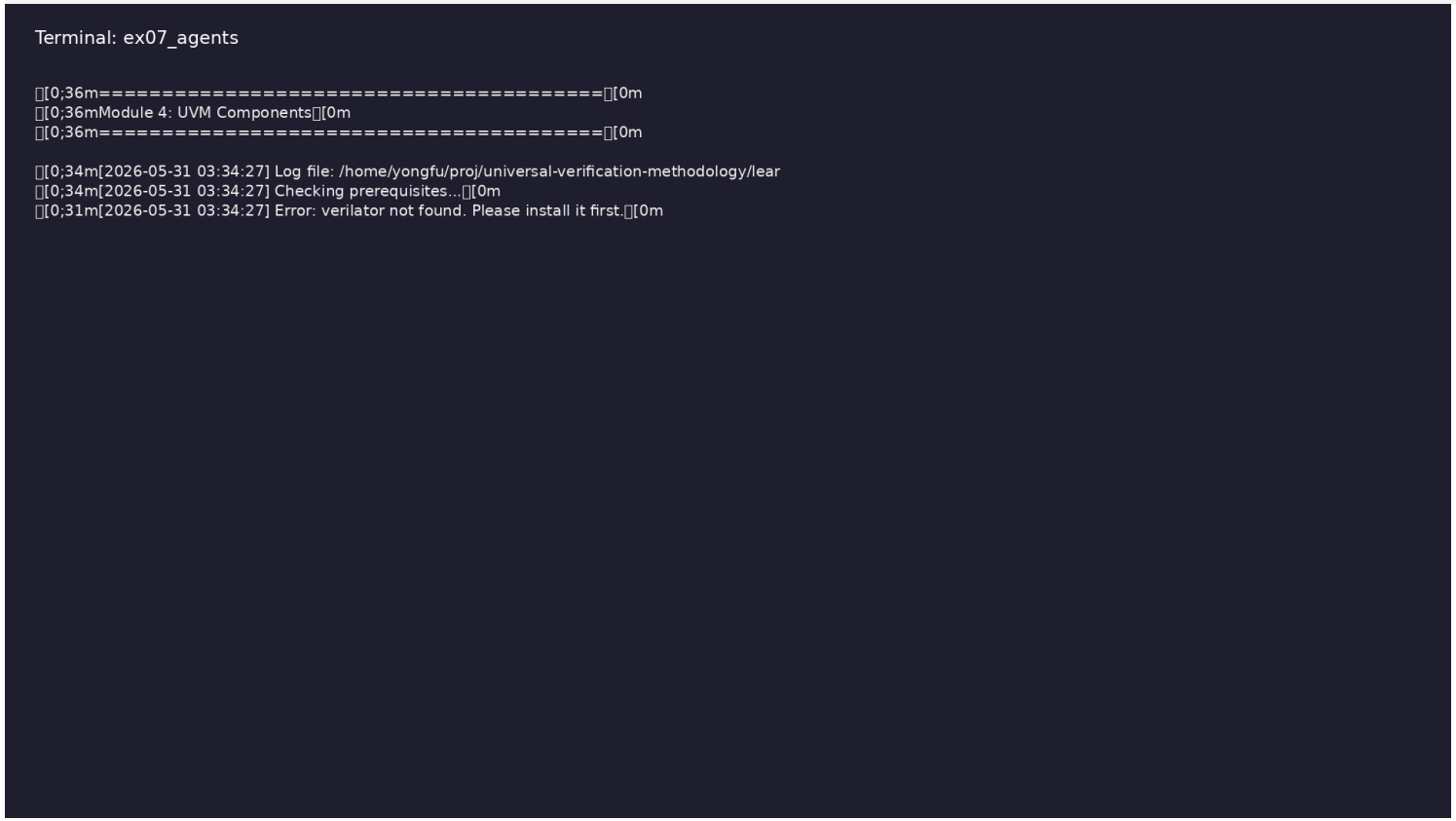


Example 7: Complete Agents

- CompleteAgent: Contains driver, monitor, and sequencer
- Agent Configuration: Configurable as active or passive
- Component Integration: Integrating all agent components
- Agent Testing: Testing complete agent functionality

Demo: Complete Agents

```
$ ./scripts/module4.sh --agents
```



Practice & assessment

What you should know (1/2)

- Design and implement transaction classes
- Create drivers for DUT stimulus generation
- Implement monitors for DUT observation
- Build sequencers and sequences for test generation
- Use TLM for component communication
- Create scoreboards for result checking

What you should know (2/2)

- Build complete agents with all components integrated
- Apply UVM component patterns in testbenches

Exercises

- Transaction Modeling
- Driver Implementation
- Monitor Implementation
- Sequencer and Sequences
- TLM Communication

Exercises

- Scoreboards
- Complete Agents

Assessment checklist

- Can design and implement transaction classes
- Can create drivers for DUT stimulus
- Can implement monitors for DUT observation
- Can build sequencers and sequences
- Can use TLM for component communication
- Can create scoreboards for result checking

Summary & next steps

- Build complete UVM agents with driver, monitor, and sequencer
- Complete module4/CHECKLIST.md
- Review module4/EXAMPLES.md and run each lab
- Next: Next module in course